

COMP90038 Algorithms and Complexity

Introduction and Welcome

Junhao Gan

Week 1 Lecture 1

Semester 1, 2026

In this subject, we will talk about:

- Data structures, including *arrays*, *linked lists*, *stacks*, *queues*, *trees*, *priority queues* and *graphs*
- Algorithms for various problems, including *sorting*, *searching*, *string manipulation*, *graph manipulation*, and more
- Algorithmic techniques, including *brute-force*, *decrease-and-conquer*, *divide-and-conquer*, *dynamic programming* and *greedy approaches*
- Analytical and empirical assessment of algorithms
- Complexity classes (if time permits)

- Anany Levitin. *Introduction to the Design and Analysis of Algorithms*. 3rd Edition, Pearson, 2012.

Also available on the Canvas, under “Reading Resources”:

- Steven Skiena. *The Algorithm Design Manual*. Springer, 2008.

Lecturer and subject coordinator: Junhao Gan

Research Interests:

Small-and-Sweet algorithms for solving problems on *massive data*.

- “Small”: the algorithms must be **light-weighted**, **relatively easy-to-implement** and **run fast in practice**.
- “Sweet”:
 - the algorithms are expected to **beat** or **match** the **state-of-the-art theoretical bounds**.
- “massive data”:
 - the datasets we deal with can be massively large, sometimes they may not fit in the main memory or even not in a single machine.

Subject Coordinator

My research covers:

- Algorithms (deterministic, randomized, approximate),
- Data Structures,
- Graph Theory,
- Computational Geometry,
- Combinatorial Optimisation,

on various computational models: Word RAM model, External Memory model, Massively Parallel Computation model, Streaming model, and etc.

I am the coach of competitive programming contests at University of Melbourne.

I coached five teams qualified to the ICPC World Finals in 2021, 2022, 2023, 2025 and 2026.

Head tutor: Zuqing Li

Tutors (ordered alphabetically by last names):

- Phillip Cervanjak
- Zhuowen Chen
- Xiaoyi Liu
- Kwun Ming Pang
- Yixiang Wang
- Yuxiang Wang

Other support is provided by your classmates, for example via the **ED Discussion Board in Canvas** (LMS).

The Canvas page is our notice board, repository, and discussion forum.

Lectures and Tutorials

We have a 2-hour lectures in the JH Michell Theatre on Fridays.

We provide **lecture capture** which is useful for revisiting points from a lecture; and it can, **sort of**, be a substitute for the lecture proper, even if it is a poor one.

Tutorials start in Week 2.

Next Monday (March 9) is a public holiday. There will be no tutorials on next Monday. Please try to attend the tutorial in other time slots next week.

Consultation. 2pm - 3pm Friday in Student Hub, Level 3, Melbourne Connect

- **10 Weekly Quizzes:** each worth 1% and in total worth 10%
- **Assignment 1:** due on 23:59pm April 16 (Week 7), worth 15%
- **Assignment 2:** due on 23:59pm May 22 (Week 11), worth 15%
- **A 3-hour final exam:** worth 60%

Hurdles: To pass the subject you must obtain:

- at least 30/60 in the written exam and
- at least 50/100 overall.

All these details, and more, can be found on Canvas, under “Subject Overview”.

Time Commitment

For the 12 weeks of semester, expect to spend

- 34 hours in lectures (2×12) + tutorials (1×10)
- 30 hours on assignments (15 hours for each)
- 24 hours of reading and reviewing (2 hours per week)
- 24 hours of tutorial preparation (2 hours per week)
- 8 hours on quizzes and discussion (40 mins per week)

On average: 10 hours per week

The commitment is **well worth it**:

Knowledge of algorithms is essential for any computing professional.

It expands your mind, improves complexion, and contains all the minerals and vitamins essential for developing boundless wisdom.

Assumed Knowledge

There are **two diagnostic quizzes** for Week 1

- not compulsory, but to help you work out how well you know the assumed background knowledge
- **Mathematics**: sets, relations, functions, recurrence relations
- **Programming**: arrays, records, linked lists, dictionaries, functions, procedures, formal and actual parameters, parameter passing, return-values, pointers/references

See the **Reading Resources** page on Canvas which has some pointers to online resources that may help with the above background knowledge.

Our previous head tutor had made a set of slides about [some basic maths knowledge](#) that would be useful in the subject. You may find it in the Week 1 module in Canvas.

How to Succeed

Understand the material (**by doing**), **DO NOT** just memorize it.

If you fall behind, try to catch up as fast as possible.

Don't procrastinate. **Start assignments early.** Put in the necessary time.

Attempt the tutorial questions **every** week, **before** you attend the tutorial, if at all possible.

How to Succeed

Support the learning of your fellow students and expect their support, in class and through the LMS discussion board.

We are all on the same learning journey and have the same goal.

Participate in the discussions on the subject's LMS site and check regularly for announcements.

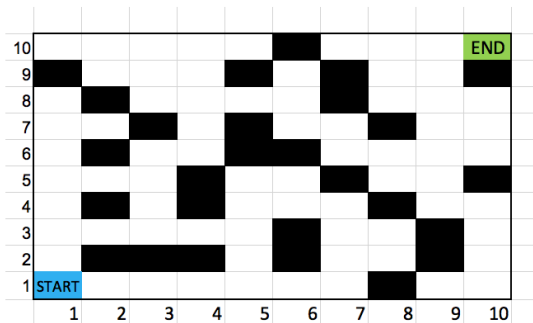
Over to You—Introductions

Please introduce yourself to your neighbours.

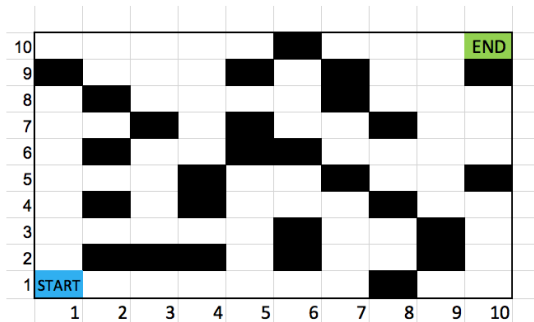
Tell them where you are from, what degree program you are enrolled in, which languages or programming languages you speak, what music you listen to, whatever.

Over to You—A Maze Problem

A maze (or labyrinth) is contained in a 10×10 rectangle; rows and columns are numbered from 1 to 10.



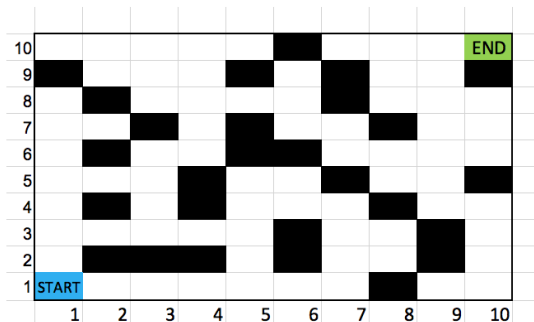
Over to You—A Maze Problem



It can be traversed along rows and columns: up, down, left, right.

The starting point is (1,1), the goal point is (10,10).

Over to You—A Maze Problem



These points are obstacles that you cannot travel through:

(3,2) (6,6) (2,8) (5,9) (8,4) (2,4) (6,3) (9,3) (1,9) (3,7)
(4,2) (7,8) (2,2) (4,5) (5,6) (10,5) (6,2) (6,10) (7,5) (7,9)
(8,1) (5,7) (4,4) (8,7) (9,2) (10,9) (2,6)

Goal: Find a path through the maze.

What is a Problem?

The Oxford Dictionary says: “doubtful or difficult question or task.”

In computer science we use the term like that too, but there is a more technical concept of **algorithmic problem**.

We usually want to find a **single generic solution** to *a bunch of similar questions*.

For example, the “maze problem” is to come up with a **mechanical solution** to *any* particular maze.

So to us, a “problem” usually has *many* (input) instances, sometimes *infinitely many*.

Example Definitions of the Maze Problem

Consider a maze contained in size 10×10 rectangle; rows and columns are numbered from 1 to 10. One can move along rows and columns: up, down, left, right.

Given:

- a start point coordinate,
- a goal point coordinate, and
- a list of specified obstacle coordinates,

decide if there exists a path from the start point to the goal point.

Question 1: Are there infinitely many input instances? And why?

Question 2: How about we further parameterize the size of the rectangle as $\alpha \times \alpha$, where α is an arbitrary positive integer?

Algorithmic Problems

So a **problem** in computer science typically means a family of **instances** of a general problem.

An **algorithm** for the problem has to work for all possible (input) instances.

Example: The **sorting** problem—an instance is a set of *comparable* items.

Example: The **graph colouring** problem—an instance is a graph.

Example: **Equation solving** problems—an instance is a set of, say, linear equations.

What is an **Algorithm**?

The dictionary: “process or rules for (esp. machine) calculation etc.”

A finite sequence of instructions

- No ambiguity, and each step is precisely defined
- Should work for all (well-formed) input
- Should finish in a finite (reasonable) amount of time

The (single) description of a process that will transform *arbitrary input* to the *correct output*—even when there are infinitely many possible inputs.

Like a cookbook recipe?

Sort of, but more like a **method**, a systematic approach that works for **any** instance.

For example, the **Depth-First Search (DFS)** algorithm can solve the Maze Problem (with infinitely many input instances).

What is an Algorithm?

Not long ago, “algorithm” was synonymous with “*numeric algorithm*”.

Mathematicians had found many clever algorithms for all sorts of numeric problems.

In BC 300, the ancient Greek mathematician Euclid proposed the first algorithm for calculating the **greatest common divisor (GCD)** of two given positive integers m and n with $m \geq n$.

The *Euclidean Algorithm* $\text{gcd}(m, n)$ works as follows:

Step 1: If $n = 0$, **return** the value of m as the answer and **stop**.

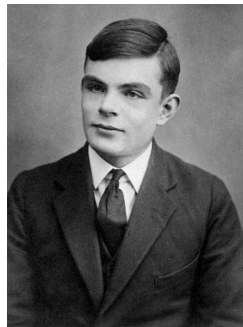
Step 2: $r \leftarrow m \bmod n$.

Step 3: $m \leftarrow n$ and $n \leftarrow r$;
go to Step 1.

Nowadays, numeric algorithms are just a small part of the syllabus in an algorithms course.

At the time of Alan Turing's birth, a “computer” was a human employed to do tedious numerical calculations.

Legacy: “Turing machine”, the “Church-Turing thesis”, “Turing reduction”, the “Turing test”, the “Turing award”



One of Turing's great accomplishments was to put the **concept** of an algorithm on a **firm mathematics foundation** and to establish that certain important problems **do not have algorithmic solutions**, e.g., the **Halting Problem**.

In this subject, we are *only* interested in problems that have algorithmic solutions.

However, amongst those, there are many that provably do not have *efficient* solutions.

Towards the end of this subject, if time permits, we will discuss *complexity theory* briefly—this theory is concerned with the inherent “hardness” of problems.

Why Study Algorithms?

Computer science is increasingly an enabler for other disciplines, providing *useful tools* for these.

Algorithmic thinking is relevant in *life sciences*, in *engineering*, in *linguistics*, in *chemistry*, etc.

Today computers allow us to solve problems whose size and complexity is vastly greater than what could be done a century ago.

The use of computers has **changed the focus of algorithmic study completely**, because algorithms that work well for a human (small scale) usually do not work well for a computer (big scale).

Why Study Algorithms and Their Complexity?

To collect a number of useful problem solving tools.

To learn, from examples, strategies for solving computational problems.

To be able to write robust programs whose behaviour we can reason about.

To develop analytical skills.

To learn about the inherent difficulty of some types of problems.

Problem Solving Steps

- Understand the problem
- Decide on the computational means (sequential/parallel, exact/approximate)
- Decide on method to use (algorithm design technique or strategy, use of randomization)
- Design the necessary data structures and algorithm
- Check for correctness, trace example input
- Evaluate analytically (time, space, worst case, average case)
- Code it
- Evaluate empirically

What We Will Study

Algorithm analysis

Important algorithms for various problems, primarily

- Sorting
- Searching
- String processing
- Graph algorithms

Approaches to algorithm design

- Brute force
- Decrease and conquer
- Divide and conquer
- Transform and conquer

Study Tips

Before the lecture, as a minimum make sure you have read the introductory section of the relevant chapter.

Always read (and work) with paper and pencil ready; run algorithms by hand.

Always have a go at the tutorial exercises; this subject is very much about **learning-by-doing**.

After the lecture, re-read and consolidate your notes.

Identify areas not understood and use the LMS Discussion Forum.

Rewrite your notes if that helps.

Things to Do in the First Two Weeks

Get the textbook, read Chapter 1, and skim Chapter 2.

Make sure you have a unimelb account.

Visit the COMP90038 LMS pages and check the weekly schedule and any new announcements (at least *twice per week*).

Use the LMS Discussion Board; for example, if you are interested in forming a study group with like-minded people, the Discussion Board is a useful place to say so.